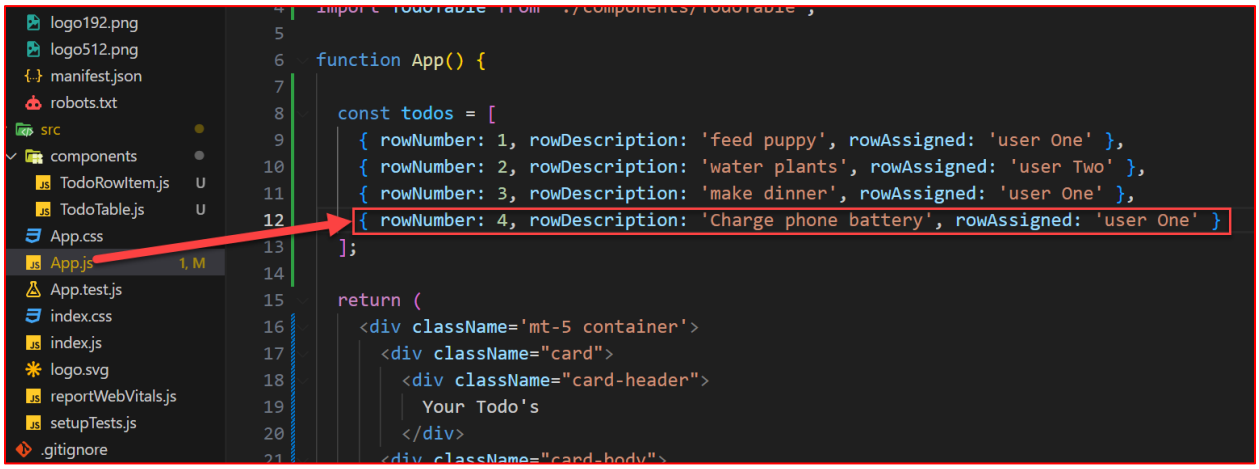
Rendering React Lists Dynamically

As we continue to build our TodoTable and TodoRowItems, a slight bug has surfaced that could become problematic as our to-do list grows. The issue lies in our code explicitly printing only three to-do row items.

The Problem

For illustration, let's go back to our "App.js" file and create an entirely new to-do item. Let's say this to-do is row number 4, with a description of "charge phone battery" and assigned to "user one." Now, if we save this change and run our application with `npm start`, then refresh the browser, you'll see that we're still only printing three items.



The Cause

This behavior is due to our "ToDoTable" specifically being told to print only three items. However, what we want to do is create a new row item for each element within our array of to-dos.

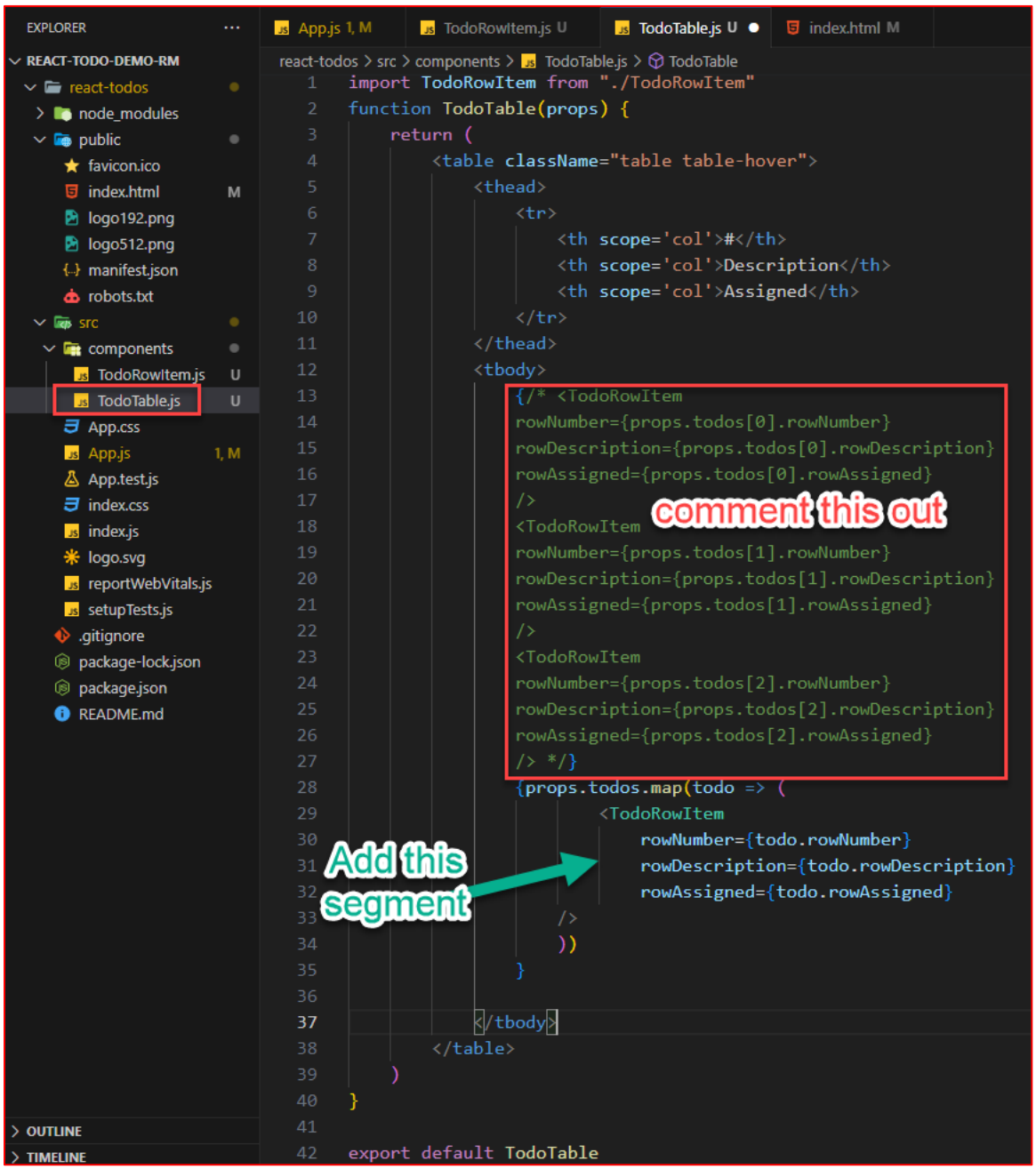
The Solution

To achieve this, we'll use the JavaScript `.map` method, which will create a new array that renders a row item for each element in our to-dos. Here's how to do it:

1. **Erase the Static Rows**: Delete the hard-coded rows.
2. **Use the `.map` Method**: In its place, insert:

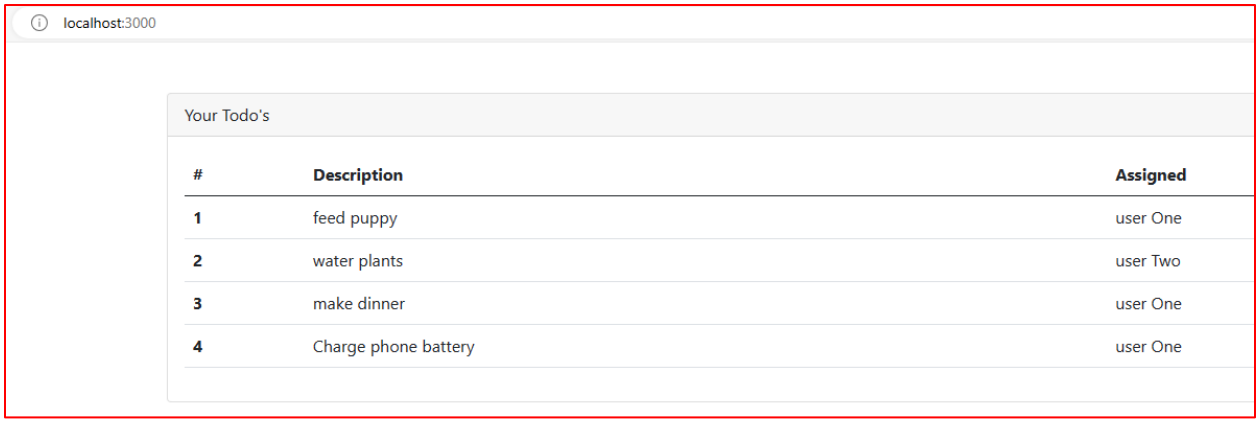
```
{props.todos.map(todo => (<TodoRowItem rowNumber={todo.rowNumber}
                          rowDescription={todo.rowDescription}
                          rowAssigned={todo.rowAssigned}/>))}
```

This code will iterate over our to-dos array and render a "TodoRowItem" for each element. The first time it runs, "todo" will represent the zero index, the second time the first index, and so on.



Result

With these changes, "ToDoRowItem" will render four separate times since we have four elements in our to-dos. Refreshing our application, we now see four tasks: "Feed puppy," "Water plants," "Make dinner," and "Charge phone battery".



Why This Matters

This pattern of rendering components based on the number of items in a list is a fundamental aspect of React. We use this functionality every single time we need to render multiple components from an array. Understanding this technique is crucial, as it allows us to create more dynamic and responsive interfaces.

Summary

In this lesson, we've learned how to dynamically create components based on the number of items within an array. This method is a vital part of building scalable and flexible React applications.